
SmartCliff
CAREER MOBILITY SOLUTIONS
We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

SmartCliff
Career Mobility Solutions

SQL BASICS

DAY- MONTH - YEAR

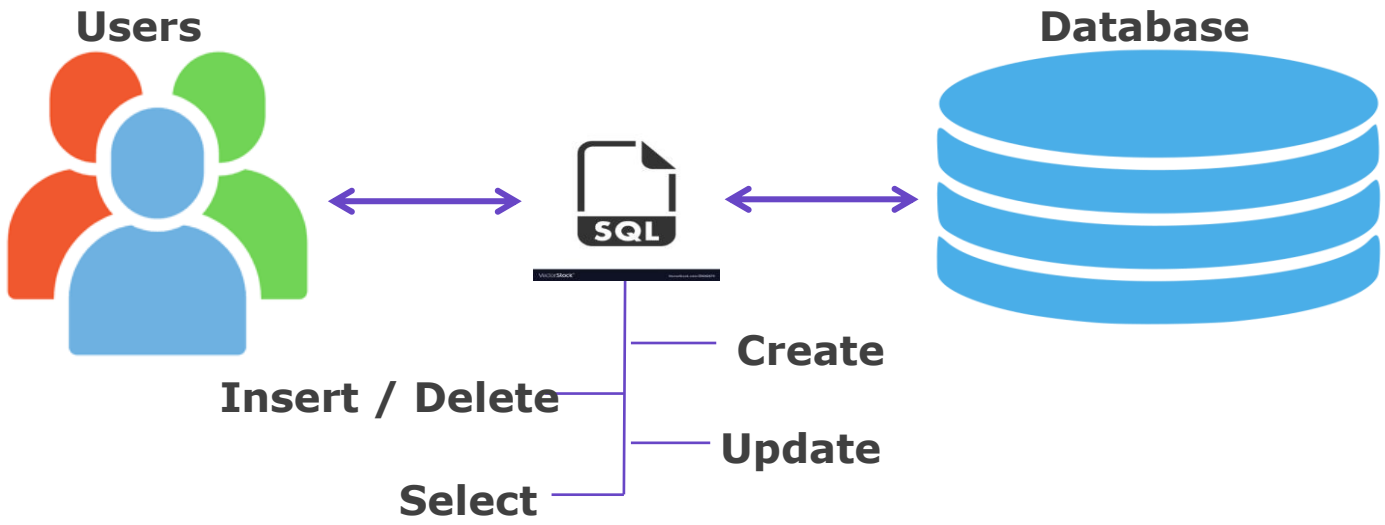
Contents

Contents

SQL Data Definition and Data Types

Introduction to SQL

- **Structured Query Language** (SQL) is a standard programming language specifically designed for **managing and manipulating relational databases**.
- SQL closely resembles the **English language,** making it **easier to understand and use.**
- It allows users to **create, read, update, and delete** (often abbreviated as CRUD) data stored in a structured format using tables.



Key Categories of SQL

1. **Data Query Language (DQL)** – Used for retrieving data from the database.
2. **Data Manipulation Language (DML)** – Used for inserting, updating, and deleting data within tables.
3. **Data Definition Language (DDL)** – Used to define, modify, or remove the structure of database objects such as tables, views, and schemas.
4. **Data Control Language (DCL)** – Used to manage access permissions and security levels for users in the database.
5. **Transaction Control Language (TCL)** – Used to manage transactions, ensuring data consistency and integrity across multiple operations.

General Rules for Naming Databases and Tables in SQL

When **naming databases or tables**, it's important to follow a **set of standard conventions** to **avoid errors** and ensure **compatibility across systems**.

- **Characters Allowed:** Letters (A–Z, a–z), Digits (0–9), Underscore (_), Dollar sign ($).
- **Cannot Use Reserved Keywords** (like SELECT, TABLE, etc.) unless enclosed in backticks

  Example: CREATE TABLE `select` (id INT); -- valid with backticks
- **Length Limit:** Maximum 64 characters for table and database names.
- **Case Sensitivity:** Database names are case-sensitive on Unix/Linux, but case-insensitive on Windows.
- **Must Start with a Letter or Underscore or Dollar sign**
- **No Spaces** or **Special Symbols** (like @, #, !, etc.) unless **enclosed in backticks**

  Example: CREATE TABLE `my table!` (id INT); -- works but discouraged

DataTypes

- Before exploring the **different categories of SQL** in detail, it's **important** to first understand the concept of data types as follows.
- In SQL, Each **column in a database** table is required to have a **name** and its respective **data type**.
- The **data type tells the database what kind of data** can be **stored** in a **column.**

Main Categories of Data Types in MySQL:

- **Numeric Data Types :** Used to **store numbers** (e.g., INT, FLOAT, DECIMAL).
- **String Data Types** : Used to **store text or characters** (e.g., VARCHAR, CHAR, TEXT).
- **Date and Time Data Types** : Used to **store dates, times, or both** (e.g., DATE, TIME, DATETIME, TIMESTAMP).

DataTypes - Numeric

Type	Storage (Bytes)	Minimum Value Signed	Minimum Value Unsigned	Maximum Value Signed	Maximum Value Unsigned
TINYINT	1	-128	0	127	255
SMALLINT	2	-32768	0	32767	65535
MEDIUMINT	3	-8388608	0	8388607	16777215
INT	4	-2147483648	0	2147483647	4294967295
BIGINT	8	-2^{63}	0	$2^{63}-1$	$2^{64}-1$
DECIMAL(size,d), NUMERIC(size,d)	Varies	Total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter. The maximum number for size is 65. The maximum number for d is 30. The default value for size is 10. The default value for d is 0.			
FLOAT(p)	4/8	Uses the p value to determine whether to use FLOAT or DOUBLE for the resulting data type. If p is from 0 to 24, the data type becomes FLOAT(). If p is from 25 to 53, the data type becomes DOUBLE()			

DataTypes – String

Types	Description	Categories	Range
CHAR	Contains non-binary strings. Length is fixed as you declare while creating a table. When stored, they are rightpadded with spaces to the specified length.	Trailing spaces are removed.	The length can be any value from 0 to 255.
VARCHAR	Contains non-binary strings. Columns are variable-length strings.	As stored.	A value from 0 to 255 before MySQL 5.0.3, and 0 to 65,535 in 5.0.3 and later versions.
BINARY	Contains binary strings.	-	0 to 255
VARBINARY	Contains binary strings.	-	A value from 0 to 255 before MySQL 5.0.3, and 0 to 65,535 in 5.0.3 and later versions.

DataTypes – String

Types	Description	Categories	Range
BLOB	Large binary object that containing a variable amount of data. Values are treated as binary strings. You don't need to specify length while creating a column.	TINYBLOB	Maximum length of 255 characters.
		MEDIUMBLOB	Maximum length of 16777215 characters.
		LONGBLOB	Maximum length of 4294967295 characters
TEXT	Values are treated as character strings having a character set.	TINYBLOB	Maximum length of 255 characters.
		MEDIUMBLOB	Maximum length of 16777215 characters.

DataTypes – Data and Time

Types	Description	Display Format	Range
DATE	Use when you need only date information.	YYYY-MM-DD	1000-01-01’ to '9999-12-31
TIME	Use when you need only time information.	HH:MM:SS	-838:59:59' to '838:59:59
DATETIME	Use when you need values containing both date and time information.	YYYY-MM-DD HH:MM:SS	1000-01-01 00:00:00' to '9999-12-31
TIMESTAMP	Values are converted from the current timezone to UTC while storing, and converted back from UTC to the current time zone when retrieved.	YYYY-MMDD HH:MM:SS	1970-01-01 00:00:01 UTC to 2038-01-19 03:14:07 UTC

Quiz

1) What is the maximum length of a CHAR(10) column?

a) 5 characters

b) 10 characters

c) Unlimited characters

d) Depends on the database size

Answer : Option b)

Quiz

2) Which of the following data types is used to store date and time values?

a) TEXT

b) DATETIME

c) VARCHAR

d) NUMERIC

Answer : Option b)

Quiz

3) In SQL, DECIMAL(6,2) means:

a) 6 total digits, 2 after decimal

b) 6 decimal places, 2 integers before decimal

c) 6 digits after decimal

d) None of the above

Answer : Option a)

Quiz

4) Which data type would be best for storing a person's age?

a) CHAR

b) DATE

c) INT

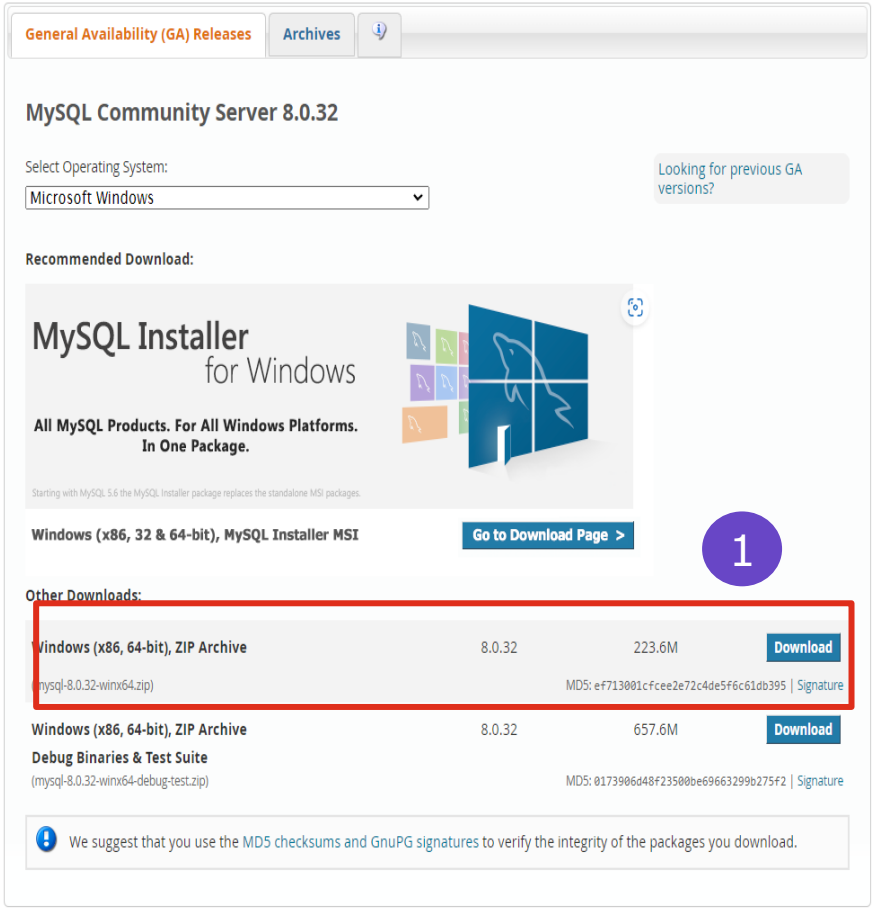
d) TEXT

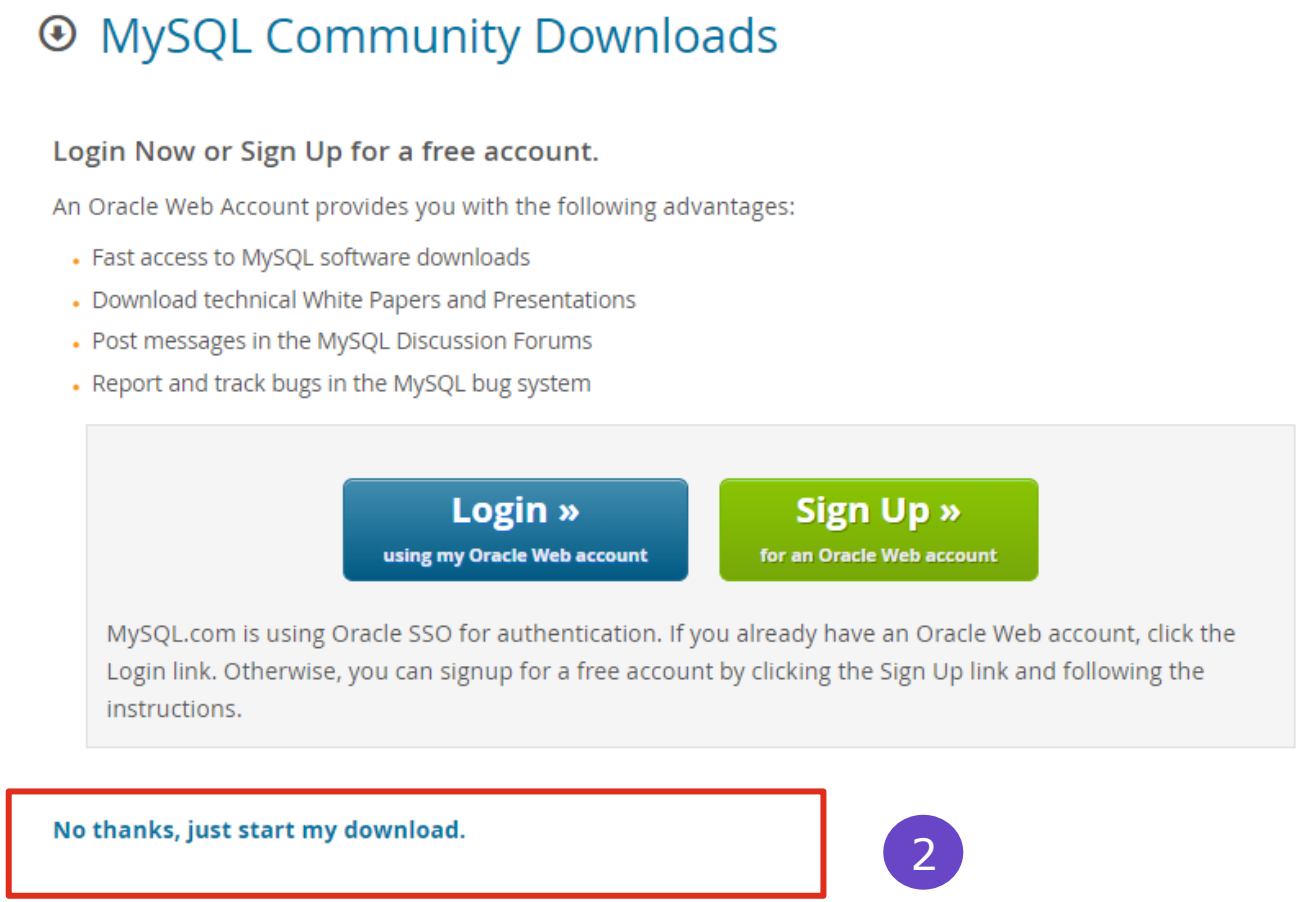
Answer : Option c)

MySQL Installation

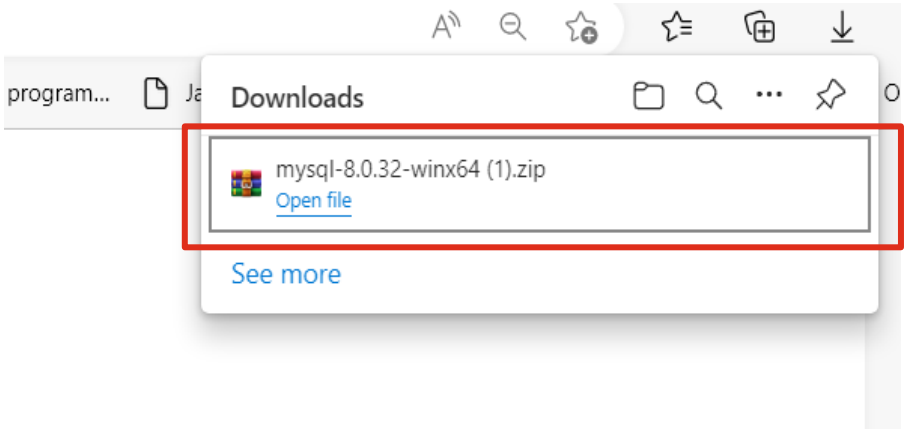
Installation Steps

- Download the latest Mysql Community server : [MySQL :: Download MySQL Community Server]

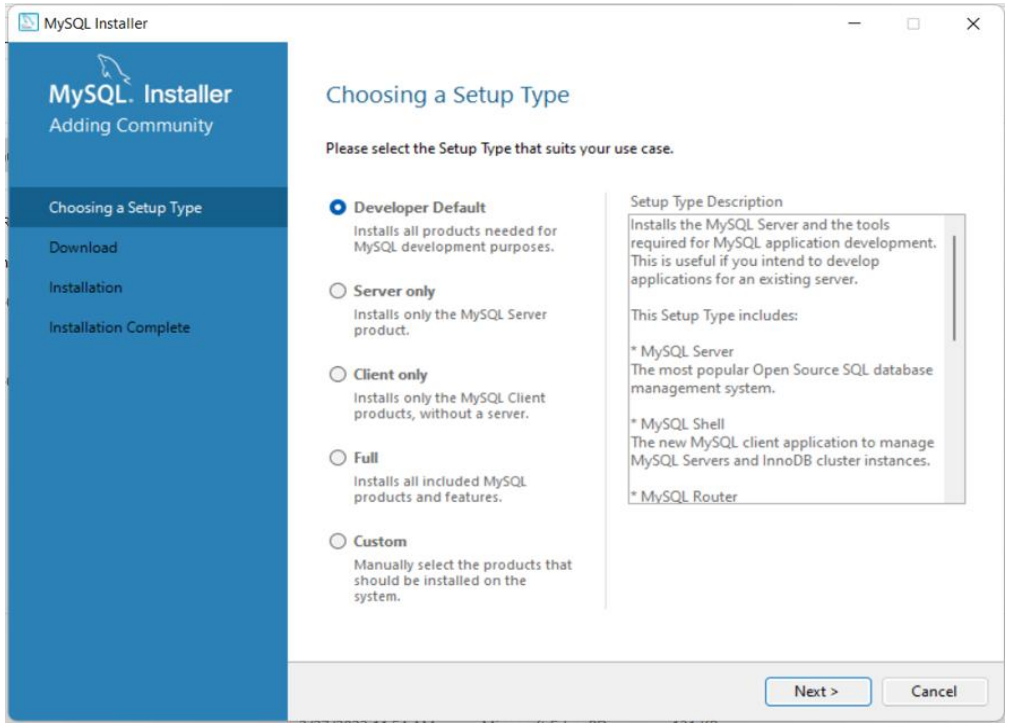




Installation Steps



3

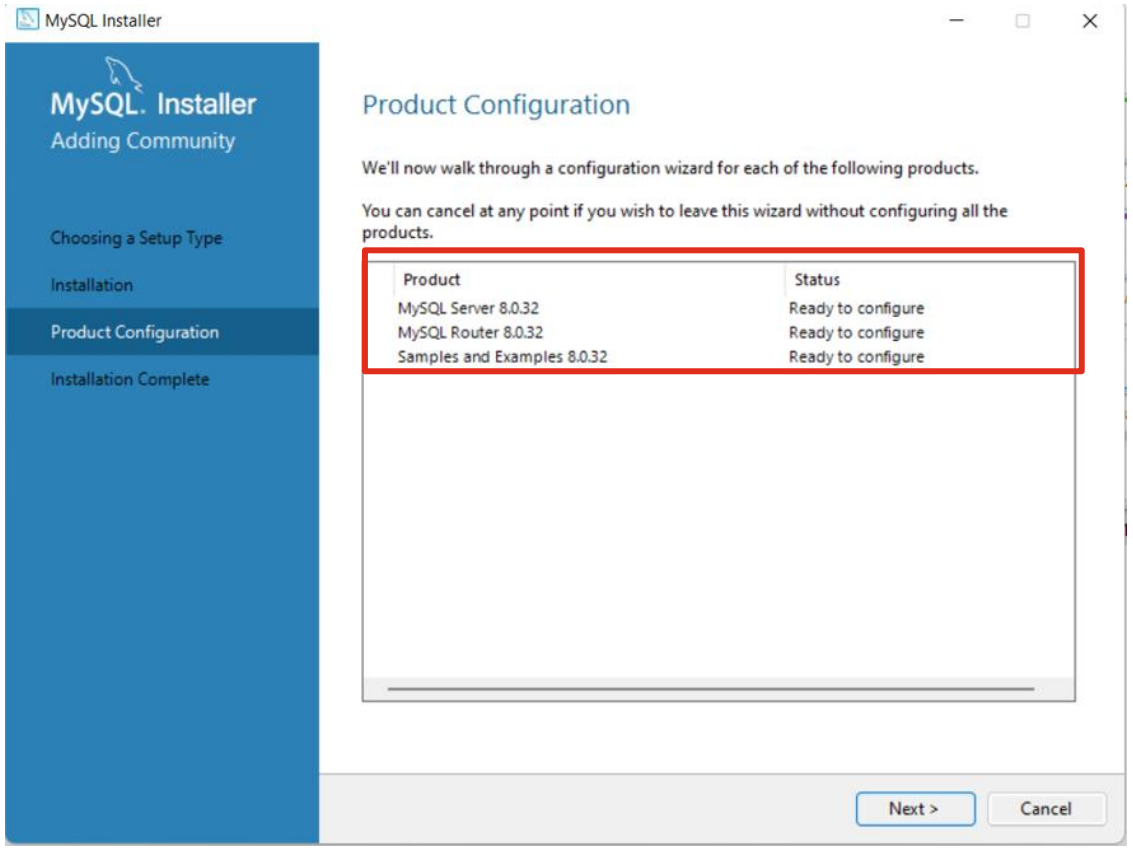


4

Follow the steps of my-sql installer community msi installer and accept the defaults to install my SQL

Installation Steps

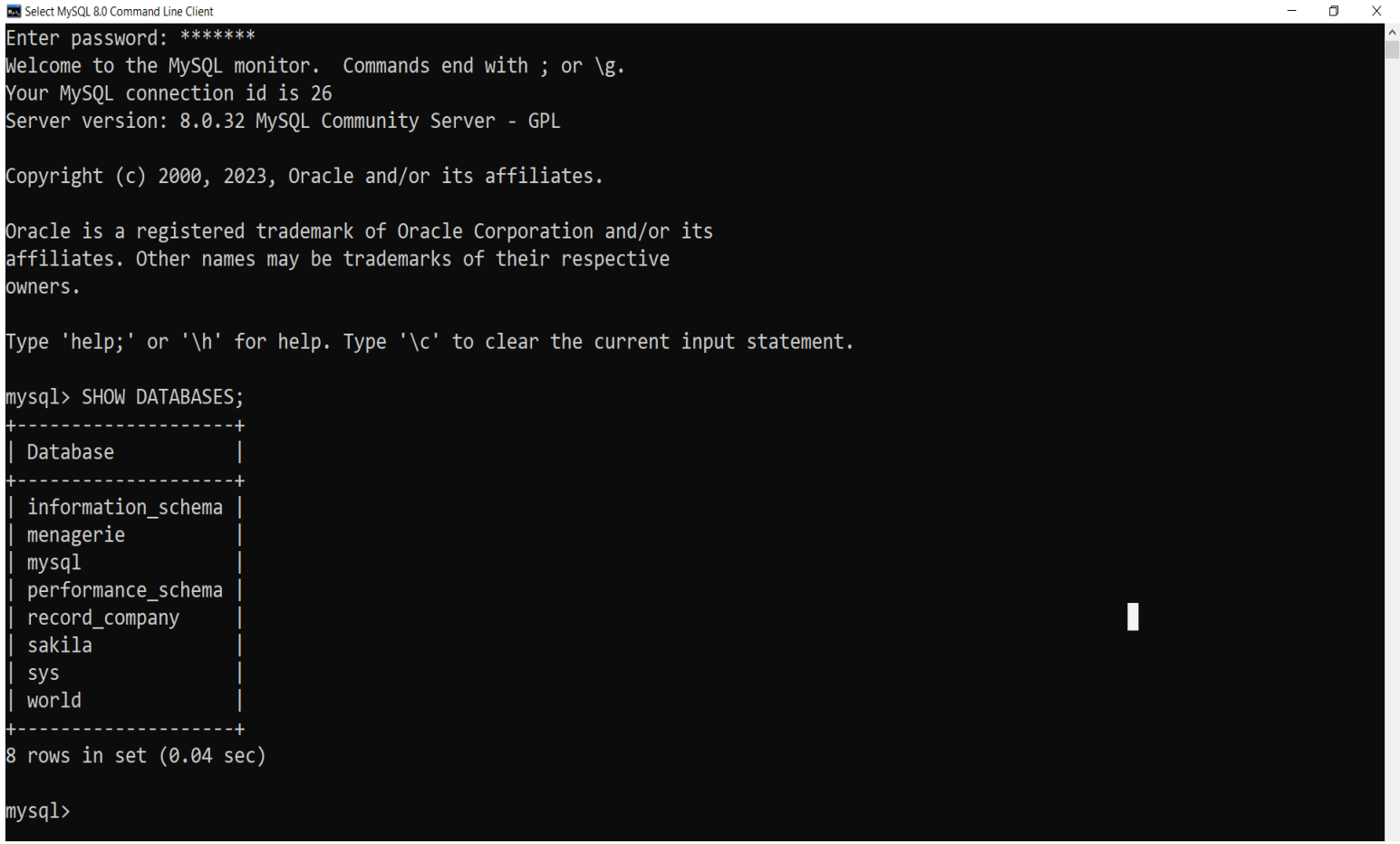
- During Installation ,below Product Configuration will take place for use,



Accessing the Database

Environment for communication:

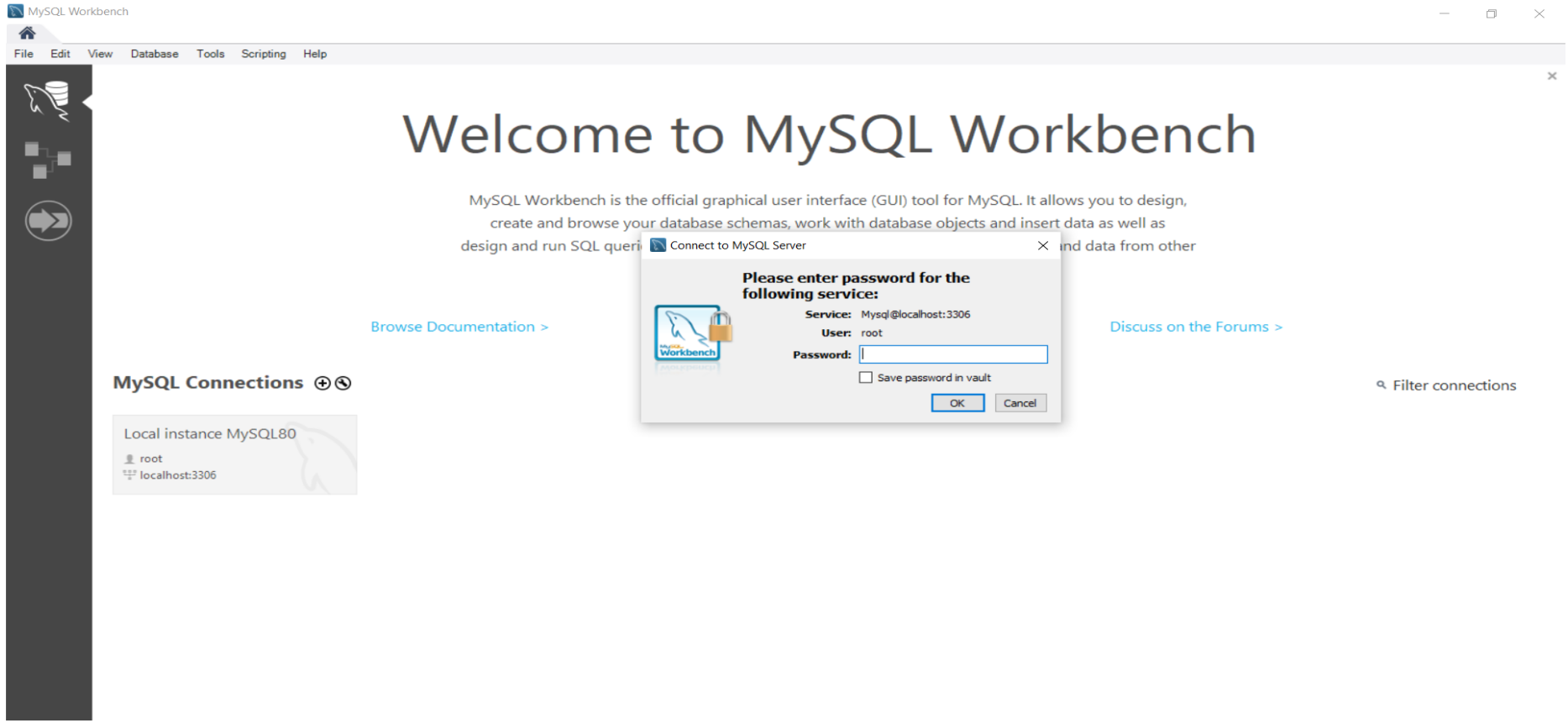
- MySQL 8.0 command line client used to communicate with the MySQL database.



Accessing the Database

Environment for communication:

- MySQL 8.0 Workbench – GUI tool used to communicate with the MySQL database.



Data Definition Language (DDL)

Introduction

- **DDL (Data Definition Language)** is a **subset of SQL** used to **define, create,** and **modify** the **structure of database objects** such as **tables, schemas, indexes, and views.**
- It defines **how the data** is **organized** in the **database, rather than managing the data itself.**
- All **DDL commands** are **auto-committed.** This means any **changes** made using **DDL are automatically saved permanently** in the **database cannot be retained or rolled back.**

DDL Commands

Command	Purpose
CREATE	Used to create a new object in the database (e.g., table, view, index).
ALTER	Used to change the structure of an existing object (e.g., add or remove a column in a table).
TRUNCATE	Used to remove all records from a table quickly, while keeping its structure.
RENAME	Used to change the name of an existing table or other database object.
DROP	Used to delete database objects completely (e.g., remove a table or view).

DDL Commands : Create

- The CREATE command is used to **create new database objects** like Tables, Databases, Views, Indexes, Schemas, Procedures, etc.

Syntax :

```
CREATE DATABASE databasename;
```

Example:

```
CREATE DATABASE University;
```

Syntax:

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ……
    columnn datatype,
);
```

Example:

```
CREATE TABLE Student (
    StudentID VARCHAR(10),
    Name VARCHAR(50),
    Email VARCHAR(50),
    DOB DATE,
    Major VARCHAR(20)
);
```

DDL Commands : Alter

- The **alter command** is used to **modify the structure** of the table.
- Modifying the structure includes:
  - Add Columns
  - Drop Columns
  - Modify Columns
  - Rename Columns
  - Add Constraints
  - Drop Constraints
  - Modify Constraints (Drop the old constraint and Re-add it with a new name)

DDL Commands : Alter- Add Columns

- Use the **ADD COLUMN** option in the ALTER TABLE command.

Syntax:

ALTER TABLE *table_name* ADD COLUMN *column_name datatype*;

ALTER TABLE *table_name* ADD COLUMN

(column1_name data type, column2_name data type…);

Examples:

```
ALTER TABLE Student ADD COLUMN Phone VARCHAR(15);

ALTER TABLE Student ADD COLUMN (Address VARCHAR(100), Gender CHAR(1));

ALTER TABLE Student ADD COLUMN DepartmentID VARCHAR(10);
```

DDL Commands : Alter- Drop Columns

- Use the **DROP COLUMN** option in the ALTER TABLE command.

Syntax:

ALTER TABLE *table_name* DROP COLUMN *column_name*;

Example:

```
ALTER TABLE Student
DROP COLUMN Phone;
```

DDL Commands : Alter- Modify Columns

- Used the **MODIFY COLUMN** option in the **ALTER TABLE** command to **change the definition of the column**. The column **data type, length , default value, constraints** can be modified.

Syntax:

ALTER TABLE *table_name*

MODIFY COLUMN *column_name datatype*;

Example:

ALTER TABLE Student

MODIFY COLUMN Address *varchar(10)*;

DDL Commands : Alter- Rename Columns

- The **RENAME COLUMN** statement is used to **change the name of the column**

Syntax:

ALTER TABLE *table_name* RENAME COLUMN *old_name TO new_name;*

Example:

```
ALTER TABLE Student

RENAME COLUMN Name TO FullName;
```

DDL Commands : Rename Command

- The **RENAME** command in SQL is used to **change** the **name of a table** or **database object**.

Syntax:

RENAME table *old_name* to *new_name*;

Example:

RENAME table Student to Student_Details;

- **RENAME** is a **DDL command**, so the **change is permanent** unless explicitly reversed.
- You can only **rename existing tables**.
- Ensure the **new name** follows **naming conventions** (no reserved keywords, valid characters, etc.).

DDL Commands : Truncate Command

- TRUNCATE is used to **delete all the rows** in the table but **maintains the structure** of the table.
- **It also deallocates the space used by the data.**
- It **cannot be rolled back**.

Syntax:

TRUNCATE TABLE *table_name*;

Example:

TRUNCATE TABLE Student;

DDL Commands : Drop Command

- The **Drop command** is used to **drop an existing SQL database** or **remove the table** from the database.

Syntax :

DROP DATABASE *databasename*;

Example:

DROP DATABASE *University*;

Syntax:

DROP TABLE *table_name*;

Example:

DROP TABLE Student;

SQL Constraints

Analogy to Understand SQL Constraints

Consider the **School admission form** as follows:

- **The school sets some rules:**
  - You must fill in your name (can’t leave it blank).
  - Your roll number must be unique (no two students can have the same roll number).
  - You must be at least 5 years old to join.
  - If you don’t write your city, it will be filled in as “Unknown” by default.
- **These rules ensure that the form is filled correctly and completely.**
- Similarly, **SQL constraints** are **rules set on database columns** to make **sure the data entered** is **correct, meaningful,** and **consistent.**

SQL Constraints-Introduction

- **SQL Constraints** are **rules applied to table columns** that help **control the type of data stored in the database.**

Why Do We Need Constraints?

- To prevent missing or incorrect data
- To ensure each record is valid
- To maintain relationships between tables
- To automate basic validation

Common Types of SQL Constraints in MySQL

Constraint	Purpose
NOT NULL	Data **must be provided** (can't leave it blank)
UNIQUE	Value must be **different from all other rows**
PRIMARY KEY	Uniquely identifies each row (**NOT NULL + UNIQUE**)
FOREIGN KEY	Ensures a value **matches a key in another table**
CHECK	Value must **follow a specific rule** (like age > 18)
DEFAULT	Provides a **default value** if nothing is entered

Note : In SQL, constraints are **defined** when **creating or altering tables** via **DDL** (Data Definition Language) commands.

NOT NULL Constraints with DDL Commands

- **Ensures** that a **column** must **always contain a value**; it **cannot be left empty** (NULL).
- It is used when a **field is mandatory**, such as an ID or a name, which is **required for every record**.

Example:

```
CREATE TABLE Department (
    DepartmentID VARCHAR(10) NOT NULL,
    Name VARCHAR(50) NOT NULL,
    Building  VARCHAR(50)
);
```

In this example:

- DepartmentID and Name **must** have values for every row.
- Building can be left empty (NULL) if the information is not available.

UNIQUE Constraints with DDL Commands

- The **UNIQUE** constraint ensures that **all the values** in a column are **different** — **no two rows** can have the **same value** in that column.It can also help in **uniquely identifying** records.
- A UNIQUE column **can accept NULL values** (but only one NULL in most databases like MySQL).

Example:

```
CREATE TABLE Department (
    DepartmentID VARCHAR(10) NOT NULL UNIQUE,
    Name VARCHAR(50) NOT NULL,
    Building VARCHAR(50)
);
```

In this example:

- DepartmentID must be **unique** for every row.
- No two departments can have the same DepartmentID.
- Name and Building have no uniqueness restriction here.

PRIMARY KEY Constraints with DDL Commands

- The **PRIMARY KEY** constraint is used to **uniquely identify** each record in a table. It also ensures that the column **always has a value** — meaning it is automatically **NOT NULL**.
- A **table** can have only **one PRIMARY KEY.**
- The table containing the **primary key** is often **referred** to as the **master** or **parent table** in **relational databases.**
- A **primary key** can be made up of **one or more columns** (in which case, it's **called a composite key).**

PRIMARY KEY Constraints with DDL Commands

Example:

```
CREATE TABLE Department (
    DepartmentID VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Building VARCHAR(50)
);
```

In this example:

- DepartmentID is the **primary key**, so each department must have a unique ID and it cannot be empty.
- Name is required (NOT NULL), but not necessarily unique.
- Building is optional in this case.

FOREIGN KEY Constraints with DDL Commands

- The **FOREIGN KEY** constraint is used to **establish a relationship** between **two tables** by linking a **column in one table (Child)** to the **PRIMARY KEY in another table (Parent).**
- Ensures **referential integrity** — values in the foreign key column must exist in the referenced primary key column.
- A **single primary key** can be referenced by **multiple foreign keys from different tables.**
- Foreign key values **can be repeated** — Example: multiple courses can belong to the same department.
- If the **parent record is deleted or updated**, the **foreign key** behavior can be **controlled** using **ON DELETE** or **ON UPDATE** actions.

FOREIGN KEY Constraints with DDL Commands

Example:

```
CREATE TABLE Department (
    DepartmentID VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Building VARCHAR(50)
);
```

In this example:

- DepartmentID is the **primary key**, so each department must have a unique ID and it cannot be empty.

FOREIGN KEY Constraints with DDL Commands

Example:

```
CREATE TABLE Course (
    CourseID VARCHAR(10) PRIMARY KEY, Title VARCHAR(100) NOT NULL,
    Credits INT CHECK (Credits>0), DepartmentID VARCHAR(10),
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

In this example:

- DepartmentID is the **primary key**, so each department must have a unique ID and it cannot be empty.
- Name is required (NOT NULL), but not necessarily unique.
- Building is optional in this case.

CHECK Constraints with DDL Commands

CHECK Constraint:

- The **CHECK constraint** ensures that a **column's** value **meets a specified condition** and it can be used to **restrict values** to a **specific range**, **pattern, or set of allowed values**.
- **Validates data** based on a **Boolean condition**.
- **Rejects any value** that **does not satisfy the condition**.
- Useful for **domain-specific validations** (e.g., age limits, department codes, salary ranges).

CHECK Constraints with DDL Commands

Example:

```
CREATE TABLE Student (
    StudentID VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(50) UNIQUE, DOB DATE,
    Major VARCHAR(15) NOT NULL CHECK (Major IN ('CS', 'IT', 'Mechanical', 'Civil', 'ECE', 'EEE'))
);
```

In this example:

- Major can only take one of the six allowed values — 'CS', 'IT', 'Mechanical', 'Civil', 'ECE', 'EEE'.
- If a user tries to insert 'BioTech' or any other value not in the list, the database will **reject** it.

DEFAULT Constraints with DDL Commands

- The **DEFAULT constraint** in MySQL **automatically** assigns a **value** to a **column** if **no value is provided** during **INSERT.**
- This is useful for **optional columns** where you want a **predefined value without always specifying it.**
- **The default value** can be a **constant (string, number, date) or a function (like CURRENT_TIMESTAMP for dates).**

Example:

```
CREATE TABLE Department (
    DepartmentID VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Building VARCHAR(50) DEFAULT 'MAIN BLOCK'
);
```

In this example:

- Building VARCHAR(50) DEFAULT 'MAIN BLOCK' : If you **don't** provide a building name, it will **automatically store** 'MAIN BLOCK'.

Named Constraint in MySQL

- A **named constraint** is when you **explicitly** give a **name to a database constraint** (e.g., PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT) **instead of letting MySQL auto-generate** one.
- **This is very useful for:**
  - **Clarity & Maintainability** — Anyone reading the schema immediately knows what the constraint enforces.
  - **Easy Identification & Debugging** — MySQL error messages will show the constraint's name, making it easier to identify which rule failed.
  - **Dropping/Modifying Constraints** — You can only drop or change a constraint if you know its name.
  - **Complex Constraints** — When constraints involve multiple columns, naming avoids confusion.

Named Constraint in MySQL : Example

Example:

```
CREATE TABLE Department (
    DepartmentID VARCHAR(10),
    Name VARCHAR(50) NOT NULL,
    Building VARCHAR(50) DEFAULT 'MAIN BLOCK',
    CONSTRAINT PK_Department PRIMARY KEY (DepartmentID),
    CONSTRAINT UQ_DepartmentName UNIQUE (Name)
);
```

In this example:

- **PK_Department** → This is a ***named*** **primary key** constraint for the column **DepartmentID.**
- **UQ_DepartmentName** → This is a ***named*** **unique** constraint to ensure **no two departments have the same name.**

Named Constraint in MySQL : Example

- If already the table is created without named constraint, MySQL will **auto-generate** one like **PRIMARY (for PK)** or something like **table_name_ibfk_1 (for foreign keys).**
- If later you want to **replace that auto-generated constraint name** with **something like PK_Department, you can't directly rename it —** you'll have to:

Drop the existing constraint:

- To **rename,** you must **first remove the existing PK constraint:**

Example:

```
ALTER TABLE Department DROP PRIMARY KEY;
```

Named Constraint in MySQL : Example

Add it back with your desired name:

Example:

```
ALTER TABLE Department
ADD CONSTRAINT PK_Department PRIMARY KEY (DepartmentID);
```

- Now the **primary key constraint** is **explicitly named PK_Department.**
- If there's **an error or** if you need **to drop** it later, you can **easily reference** it by **name "**PK_Department **"**

Named Constraint in MySQL : Example

Why we name them:

1. **Easy Debugging** – If Name is duplicated, the error will say: ***exactly*** **which rule failed.**

```
Error: Duplicate entry... violates constraint 'UQ_DepartmentName'
```

2. **Easy Maintenance –** If we want to remove this constraint later**:**

```
ALTER TABLE Department DROP CONSTRAINT UQ_DepartmentName;
```

3. **Clear Purpose –** Anyone reading the schema immediately understands:
   - PK_Department → Primary key for Department
   - UQ_DepartmentName → Unique Department Name

Constraint Levels

- In MySQL, when we talk about **constraint levels**, we're usually **referring to where and how a constraint is applied in a table definition.**
- There are **two main levels**:
    1. **Column-Level Constraints :** Applied **directly** when **defining a column**; **affects only that column.**
    2. **Table-Level Constraints : Declared separately** after **all columns are listed**; can involve **multiple columns or reference another table**.

Constraint Levels : Column-Level Constraints

- **Column Level Constraints** are written **right next** to the **column definition** like below:

Example:

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,              -- Column-level PK
    Name VARCHAR(100) NOT NULL,             -- Column-level NOT NULL
    Email VARCHAR(100) UNIQUE,              -- Column-level UNIQUE
    DOB DATE,
    Major VARCHAR(50),
```

Constraint Levels : Table-Level Constraints

- **Table level Constraints** are defined after **all columns are declared**, and usually **Involve multiple columns** or **reference foreign keys** or **set complex conditions**.

Example:

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE, DOB DATE,
    Major VARCHAR(50),
    -- Table-level constraint for multiple columns
    CONSTRAINT CHK_Major CHECK (Major IN ('CS', 'IT', 'ECE', 'MECH'))
);
```

Constraint Levels

- The below table summarize which constraints can be applied at column-level and table-level in MySQL:

Constraint Type	Column-Level	Table-Level
NOT NULL	Yes	No
UNIQUE	Yes	Yes multi-column)
PRIMARY KEY	Yes	Yes (multi-column)
FOREIGN KEY	No	Yes
CHECK	Yes	Yes
DEFAULT	Yes	No

Quiz

1) Which SQL command is used to remove a table along with its data permanently?

a) DELETE

b) DROP

c) TRUNCATE

d) REMOVE

Answer : Option b)

Quiz

2) Which of the following DDL statements is used to change the structure of an existing table?

a) ALTER

b) UPDATE

c) MODIFY

d) CHANGE

Answer : Option a)

Quiz

3) What will TRUNCATE TABLE Students; do?

a) Remove the table and its structure

b) Remove all rows but keep the structure

c) Remove only specific rows

d) Reset the table to its default values without deleting rows

Answer : Option b)

Quiz

4) What does the following command do?

```
ALTER TABLE Employee ADD Salary INT;
```

a) Changes all salary values to INT

b) Adds a new column named Salary of type INT

c) Creates a new table with Salary column

d) Deletes the Salary column from the table

Answer : Option b)

Quiz

5) Which constraint allows NULL values but still enforces uniqueness?

a) PRIMARY KEY

b) FOREIGN KEY

c) UNIQUE

d) NOT NULL

Answer : Option c)

Quiz

6) Which of the following CHECK constraints is valid?

a) CHECK (Age > 18)

b) CHECK Age >= 18

c) CHECK: Age >= 18

d) CHECK = Age >= 18

Answer : Option a)

Quiz

7) Which constraint is used to enforce referential integrity between two tables?

a) PRIMARY KEY

b) UNIQUE

c) FOREIGN KEY

d) CHECK

Answer : Option c)

Quiz

8) If a column is defined as NOT NULL, it means:

a) The column can have only numbers

b) The column must have a value for each row

c) The column values must be unique

d) The column is optional

Answer : Option b)

Data Manipulation Language (DML)

Introduction

- **DML (Data Manipulation Language)** is a **subset of SQL** used to **manage** and **manipulate data stored** in **database tables.**
- It focuses on **inserting, updating, and deleting records,** but it **does not change the structure of the database itself (that's done by DDL).**
- DML Command are **not auto - committed**, it means **changes by DML command are not permanent** it can be roll back.
- DML consists of the following set of commands as follows:

Command	Purpose
INSERT	Add new rows to a table.
UPDATE	Modify existing rows in a table.
DELETE	Remove rows from a table.

Insert Command

- In **DML** (Data Manipulation Language), the **INSERT command** is used to **add new rows of data** into a table.

Syntax:

```
INSERT INTO table_name VALUES (value1,value2,value3……);

INSERT INTO table_name( column1_name,column2_name…..)

VALUES(value1,value2,value3……);
```

Example:

```
INSERT INTO student VALUES ('25CS001','Smith','25CS001@u.com','2007-11-25','CS');
```

Insert Command

- If you omit the **column list** in an **INSERT statement**, you must provide **values for every column** in the table, and they must be in the **exact order** in which the **columns were defined during table** creation.

Syntax:

```
INSERT INTO table_name VALUES (value1, value2, value3, ...);
```

Example:

```
INSERT INTO Student VALUES (1, 'John Doe', 'john@example.com', 'CS');
```

Note:

Column name list is optional of values.

Character and date values must be enclosed within single quotation.

Date must be in ‘YYYY-MON-DD’ format.

Update command

- **Update command** is used to **change or modify existing values** in a table.
- You can **update all rows** or **only a specific subset of rows** by using a **WHERE clause.**

Syntax:

```
UPDATE table SET column1=value1;
```

Example:

```
UPDATE Student SET Major = 'CS';
```

- In above example, the Update changes the Major for **every student in the table.**

Update command

- **Update** specific **records** use the **WHERE clause** to **target certain rows**

Syntax:

UPDATE table SET column1=value1 WHERE column = value;

Example:

UPDATE Student SET Major = 'IT' WHERE StudentID = 3;

- In above example ,changes the Major only for the student with StudentID = 3

Delete Command

- The **DELETE** command in SQL is used to **remove one or more rows from a table**.
- It **only deletes the data** — the **table structure remains unchanged**.

Syntax:

```
DELETE FROM table_name
```

Example:

```
DELETE FROM Student;
```

- **Removes all rows** from the **Student table** but **keeps the table structure**.

Delete Command

- Delete the specific **records** use the **WHERE clause** to **target certain rows**

Syntax:

DELETE FROM TABLE WHERE column = value;

Example:

DELETE FROM Student WHERE StudentID = 5;

- In above example , Removes only the student whose StudentID is 5.

Introduction

- **AUTO_INCREMENT** is a MySQL feature that **automatically generates a unique number** for a column each time you i**nsert a new row, without you** having to provide that **value manually.**
- Usually used for **Primary Keys** so that each **record gets a unique ID.**
- Starts at **1 by default** and **increases by 1 for every new row** (this can be changed).
- If you **delete a row, the number is not reused** (to maintain uniqueness).
- It must be used on a **numeric column** (INT, BIGINT, etc.).
- A table can have **only one AUTO_INCREMENT** column.

How it works?

```
-- Example:
CREATE TABLE Student (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(50) UNIQUE,
    DOB DATE,
    Major VARCHAR(15) NOT NULL
        CHECK (Major IN ('CS', 'IT', 'Mechanical', 'Civil', 'ECE', 'EEE'))
);
```

How it works?

1. Insert without giving StudentID:

```
INSERT INTO Student (Name, Email, DOB, Major)
VALUES ('John Doe', 'john@example.com', '2001-04-15’,
'CS’);
```

Explanation:

MySQL will **automatically assign**:

- First row → StudentID = 1
- Second row → StudentID = 2
- Third row → StudentID = 3
- …and so on.

How it works?

2. Insert while giving StudentID explicitly (optional, but usually not needed):

```
INSERT INTO Student (StudentID, Name, Email, DOB, Major)
VALUES (10, 'Alice', 'alice@example.com', '2000-08-10', 'IT');
```

MySQL accepts it if the value doesn't conflict with existing IDs, but the **next auto number** will then continue **after the highest existing ID**.

3. AUTO_INCREMENT restart:

```
-- You can restart numbering  and Next inserted student gets ID = 100
ALTER TABLE Student AUTO_INCREMENT = 100;
```

Data Query Language (DQL)

Introduction

- **Data Query Language (DQL)** is the **subset of SQL** that is used to **retrieve data from a database**.It is primarily concerned with **querying (fetching) data rather than modifying it.**
- DQL has following Command:

Command	Purpose
SELECT	Retrieve data from one or more tables.

Syntax:

```
-- To select all the columns from the table

SELECT * FROM Table name;

-- To select the specific columns from the table

SELECT column1,column2,….. From table_name;
```

SELECT Command

- The below Example, We are **retrieving the all data from the Student table** using ***** instead of specifying the column name.

Example: Retrieve all column data from student table

```
SELECT * from Student;
```

Output

StudentID	Name	Email	DOB	Major
CS001	Alice Johnson	alice@example.com	2002-05-10	CS
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS003	Ethan White	ethan@example.com	2000-12-30	CS
EC001	Charlie Brown	charlie@example.com	2003-01-20	ECE
EC004	Diana Green	diana@example.com	2002-11-03	ECE

- The below data from column 1, column 2, and so on from the table in Example2.

Example: Retrieve the specified column from the student table

```
SELECT StudentID, Name from Student;
```

Output

StudentID	Name
CS001	Alice Johnson
CS002	Bob Smith
CS003	Ethan White
EC001	Charlie Brown
EC004	Diana Green

SELECT Command

- We can also retrieve the **data from the table with alias name** as follows:

SYNTAX:

SELECT Column1 As C1, Column2 As C2 from table_name;

- Here C1,C2 are the alias name that will be displayed in the result set instead of the column name.

Example: Retrieve the columns using alias name from student table

```
SELECT StudentID as ID, Name as Student_Name from Student;
```

Output

ID	Student_Name
CS001	Alice Johnson
CS002	Bob Smith
CS003	Ethan White
EC001	Charlie Brown
EC004	Diana Green

Quiz

1) Which of the following is NOT a DML command?

a) INSERT

b) DELETE

c) UPDATE

d) CREATE

Answer : Option d)

Quiz

2) Which DML command is used to remove all records from a table but keeps the table structure intact?

a) DELETE

b) TRUNCATE

c) DROP

d) REMOVE

Answer : Option a)

Quiz

3) Which SQL command is primarily used in DQL?

a) UPDATE

b) SELECT

c) DELETE

d) CREATE

Answer : Option b)

Quiz

4) The result of a DQL command is called:

a) Execution plan

b) Query set

c) Result Set

d) Data Script

Answer : Option c)

Quiz

5) In MySQL, which keyword is used to automatically generate unique numeric values for a column?

a) AUTO_ID

b) AUTO_INCREMENT

c) AUTO_NUMBER

d) AUTO_GENERATE

Answer : Option b)

Data Control Language (DCL)

Introduction

- **DCL (Data Control Language)** is the **subset of SQL** used to **control access** and **permissions** in a **database.**
- It deals with **granting or revoking rights** to **users** or **roles** so they can **perform specific actions on database objects.**
- **DCL commands** are **security-focused;** they don't change data or structure and **permissions** can be given at **database level, table level, or even column level.**
- **Database administrators** typically manage **DCL commands.**
- DCL has following Commands:

Command	Purpose
GRANT	Give specific privileges to users.
REVOKE	Remove specific privileges from users.

GRANT Command

- GRANT command gives a **user specific privileges** to **perform actions on database objects** (tables, views, databases, etc.).

Syntax:

GRANT privilege_list ON object_name TO 'username'@'host';

Example: Gives user1 permission to read and insert data into the Student table.

GRANT SELECT, INSERT ON Student TO 'user1'@'localhost';

CREATE USER 'demo_user'@'localhost' IDENTIFIED BY '1234';

ALL PRIVILEGES

REVOKE Command

- REVOKE command **Removes previously granted privileges from a user.**

Syntax:

```
REVOKE privilege_list ON object_name FROM 'username'@'host';
```

Example: Takes away the insert permission for user1 on the Student table.

```
REVOKE INSERT ON Student FROM 'user1'@'localhost';
```

NOTE :

In MySQL, there's **no DENY** command like in SQL Server. You manage access only with GRANT and REVOKE.

Quiz

1) Which DCL command is used to assign privileges to users?

a) GRANT

b) COMMIT

c) SAVE POINT

d) ROLL BACK

Answer : Option a)

Quiz

2) Which of the following removes privileges from a user in SQL?

a) REMOVE

b) DELETE

c) REVOKE

d) DENY

Answer : Option c)

Transaction Control Language (TCL)

Introduction

- **Transaction Control Language(TCL)** is used to **manage transactions** in a **database** to ensure **data consistency and integrity**.
- A **transaction** is a **sequence of one or more SQL statements** executed as a **single unit**.
- TCL often combined with **DML commands** (INSERT, UPDATE, DELETE) **inside a transaction.**
- TCL has Following commands:

Command	Purpose
COMMIT	Save all the changes made in the current transaction permanently.
ROLLBACK	Undo the changes made in the current transaction.
SAVEPOINT	Create a point within a transaction to which you can later roll back.

COMMIT Command

- **COMMIT** command **saves all changes** made during the **current transaction permanently.**

Syntax:

```
COMMIT;
```

Example:

```
START TRANSACTION;
UPDATE Student SET Major = 'IT' WHERE StudentID = 3;
COMMIT; -- Saves the update permanently
```

ROLLBACK Command

- **ROLLBACK command** is used to **reverts all changes** made in the **current transaction** to the **last committed state.**

Syntax:

```
ROLLBACK;
```

Example:

```
START TRANSACTION;
DELETE FROM Student WHERE StudentID = 5;
ROLLBACK; -- Cancels the delete operation
```

SAVEPOINT Command

- **SAVEPOINT Command creates a point** within a **transaction** to which you can **roll back without affecting the entire transaction.**

Syntax:

```
SAVEPOINT savepoint_name;
```

Example:

```
START TRANSACTION;
UPDATE Student SET Major = 'CS' WHERE StudentID = 2;
SAVEPOINT sp1;
UPDATE Student SET Major = 'ECE' WHERE StudentID = 4;
ROLLBACK TO sp1; -- Undo only the second update
COMMIT;
```

SET TRANSACTION Command

- **SET TRANSACTION Configures transaction** properties like isolation level**.**

Syntax:

SET TRANSACTION [READ WRITE | READ ONLY];

Example:

```
SET TRANSACTION READ ONLY;
START TRANSACTION;
SELECT * FROM Student;
COMMIT;
```

Example

```
-- Start the transaction
START TRANSACTION;
-- Step 1: Update a record
UPDATE Student SET Major = 'CS' WHERE StudentID = 1;
-- Step 2: Set a savepoint
SAVEPOINT sp1;
-- Step 3: Insert a new record
INSERT INTO Student (Name, Email, DOB, Major)
VALUES ('Alice', 'alice@example.com', '2001-08-12', 'IT');
-- Step 4: Rollback to the savepoint (undo the insert only)
ROLLBACK TO sp1;
```

Example

```
-- Step 5: Commit the remaining changes (update stays)
COMMIT;
-- Step 6: Optional - Set transaction as read only
SET TRANSACTION READ ONLY;
START TRANSACTION;
SELECT * FROM Student;
COMMIT;
```

Tabular comparison of all five SQL categories

DDL vs DML vs DQL vs DCL vs TCL

Type	Full Form	Purpose	Common Commands	Example
DDL	Data Definition Language	Defines and manages database structure (schema)	CREATE, ALTER, DROP, TRUNCATE, RENAME	CREATE TABLE Student (ID INT, Name VARCHAR(50));
DML	Data Manipulation Language	Inserts, updates, and deletes data in tables	INSERT, UPDATE, DELETE	INSERT INTO Student (Name) VALUES ('John');
DQL	Data Query Language	Retrieves data from tables	SELECT	SELECT Name FROM Student;
DCL	Data Control Language	Manages access rights and permissions	GRANT, REVOKE	GRANT SELECT ON Student TO 'user1';
TCL	Transaction Control Language	Manages transactions and ensures data integrity	COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION	START TRANSACTION; UPDATE Student SET Name='Alex'; COMMIT;

Quiz

1) Which of the following is a TCL command?

a) COMMIT

b) CREATE

c) SELECT

d) ALTER

Answer : Option a)

Quiz

2) Which TCL command is used to save the changes made by a transaction permanently?

a) SAVE POINT

b) COMMIT

c) ROLLBACK

d) GRANT

Answer : Option b)

Quiz

3) Which TCL command is used to undo changes in a transaction?

a) SAVE POINT

b) COMMIT

c) ROLLBACK

d) GRANT

Answer : Option c)

Quiz

4) The SAVEPOINT command is used for:

a) Restoring data from a backup

b) Creating a point to which a transaction can roll back

c) Granting access to a user

d) Committing changes automatically

Answer : Option b)

Quiz

5) In MySQL, after executing a ROLLBACK TO SAVEPOINT, what happens to the changes after that SAVEPOINT?

a) They remain intact

b) They are undone

c) They are committed automatically

d) They are duplicated

Answer : Option b)

From ER Diagram to SQL Queries – University Database (Example)

University Database – Problem Statement (Recap)

- To Illustrate the Entity relationship diagram we have taken the application – **University Database.**

A university database system manages core academic operations by tracking students, faculty, courses, and departmental structures. The system supports critical functions like student enrollment, grade recording, course scheduling, and faculty assignment. Students register for courses offered by departments, while professors teach these courses and may supervise student research. The enrollment process links students to courses while capturing semester-specific data like grades, forming the backbone of academic record-keeping. This database must handle relationships—such as professors belonging to departments, courses having prerequisites. By centralizing this information, the database eliminates redundant data entry (e.g., duplicate student records across departments) and prevents inconsistencies (e.g., a student enrolled in a nonexistent course).

University Database – Requirements (Recap)

Complete analysis of the requirement to build the Entity relationship diagram.

- Students can register for courses, view grades, and access academic records.
- Professors can submit grades and manage courses.
- Each course has a unique course code (e.g., "CS101"), title, credit hours, and assigned department.
- Courses may have prerequisites (e.g., "CS101" must be completed before "CS201").
- Students can enroll in multiple courses per semester.
- The system tracks enrollment dates, grades, and semester information.
- Each department (e.g., "Computer Science") has a unique ID, name, and budget.
- Professors are assigned to one department, while students declare a major department.
- Professors submit grades for enrolled students.

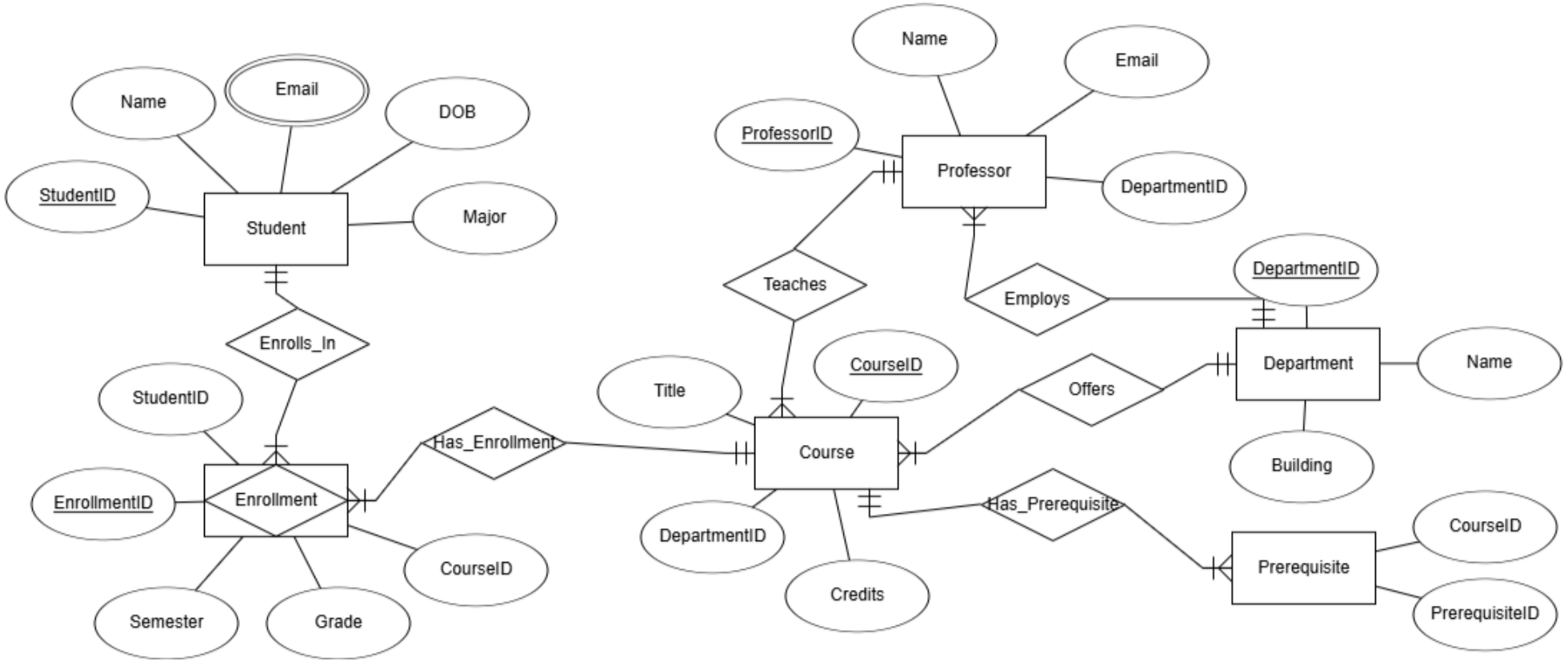
University Database – Tables Identified (Recap)

Based on the Requirement, entities or tables are defined with attribute as follows:

- Student(StudentID, Name, Email, DOB, Major)
- Department(DepartmentID, Name, Building)
- Course(CourseID, Title, Credits, DepartmentID)
- Professor(ProfessorID, Name, Email, DepartmentID)
- Enrollment(EnrollmentID, StudentID, CourseID, Grade, Semester)
- Prerequisite(CourseID, PrerequisiteID)
- Teaches(ProfessorID, CourseID)

University Database – ER Diagram (Recap)

ER Diagram define the Relationship and Cardinality between the entities or Tables as follows :



SQL Queries: Student Table

- The **Student table** stores **essential details** about **each student in the university**.
  - **StudentID:** A unique identifier for each student, set as the Primary Key.
  - **Name:** The student's full name, which cannot be left empty (NOT NULL).
  - **Email:** A unique email address for each student (UNIQUE constraint) to prevent duplicates.
  - **DOB:** Date of birth of the student.
  - **Major:** The student's field of study, restricted to one of the predefined values (CHECK constraint) — 'CS', 'IT', 'Mechanical', 'Civil', 'ECE', or 'EEE'.

SQL Queries: Student Table

- Constraints used:
  - **PRIMARY KEY** – Ensures each student record is uniquely identifiable.
  - **NOT NULL** – Prevents null values for mandatory fields (Name, Major).
  - **UNIQUE** – Ensures no duplicate email addresses.
  - **CHECK** – Restricts Major to specific valid options.

SQL Queries: Student Table

```
-- DDL Commands to CREATE the STUDENT Table
CREATE TABLE Student (
    StudentID VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(50) UNIQUE, DOB DATE,
    Major VARCHAR(15) NOT NULL CHECK (Major IN ('CS', 'IT', 'Mechanical', 'Civil', 'ECE', 'EEE'))
);
```

-- use the DESC (or DESCRIBE) command in MySQL to display its structure

```
DESC Student;
```

Output

Field	Type	Null	Key	Default	Extra
StudentID	varchar(10)	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
Email	varchar(50)	YES	UNI	NULL	
DOB	date	YES		NULL	
Major	varchar(15)	NO		NULL	

SQL Queries: Student Table

```
-- DML Commands to INSERT values into the STUDENT Table
INSERT INTO Student VALUES
('CS001', 'Alice Johnson', 'alice@example.com', '2002-05-10', 'CS'),
('CS002', 'Bob Smith', 'bob@example.com', '2001-08-15', 'CS'),
('CS003', 'Ethan White', 'ethan@example.com', '2000-12-30', 'CS'),
('EC001', 'Charlie Brown', 'charlie@example.com', '2003-01-20', 'ECE'),
('EC004', 'Diana Green', 'diana@example.com', '2002-11-03', 'ECE');
```

```
-- DQL Command to SELECT or display
all data in the STUDENT table
SELECT * FROM Student;
```

Output

StudentID	Name	Email	DOB	Major
CS001	Alice Johnson	alice@example.com	2002-05-10	CS
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS003	Ethan White	ethan@example.com	2000-12-30	CS
EC001	Charlie Brown	charlie@example.com	2003-01-20	ECE
EC004	Diana Green	diana@example.com	2002-11-03	ECE

SQL Queries: Department Table

- The **Department table** holds **information** about the **various departments in the institution**.
  - **DepartmentI**D: A unique code for each department, set as the Primary Key.
  - **Name:** The official name of the department (NOT NULL, so it must always be provided).
  - **Building:** The building where the department is located, with a default value of 'MAIN BLOCK' if not specified.
- Constraints used:
  - **PRIMARY KEY** – Ensures each department record is uniquely identified by DepartmentID.
  - **NOT NULL** – Prevents Name from being left empty.
  - **DEFAULT** – Automatically assigns 'MAIN BLOCK' to Building when no value is provided.

SQL Queries: Department Table

```
-- DDL Commands to CREATE the DEPARTMENT Table
CREATE TABLE Department (
    DepartmentID VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Building VARCHAR(50) DEFAULT 'MAIN BLOCK'
);
```

```
-- use the DESC (or DESCRIBE) command in
 MySQL to display its structure
DESC Department;
```

Output

Field	Type	Null	Key	Default	Extra
DepartmentID	varchar(10)	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
Building	varchar(50)	YES		MAIN BLOCK	

SQL Queries: Department Table

```
-- DML Commands to INSERT values into the DEPARTMENT Table
INSERT INTO Department VALUES
('D01', 'Computer Science', 'CS BLOCK'),
('D02', 'Electronics & Communication', 'ECE BLOCK');
```

```
-- DQL Command to SELECT or display
all data in the DEPARTMENT table
SELECT * FROM Department ;
```

Output

DepartmentID	Name	Building
D01	Computer Science	CS BLOCK
D02	Electronics & Communication	ECE BLOCK

SQL Queries: Course Table

- The **Course table** stores **details of courses offered** by various departments.
  - **CourseID:** Unique identifier for each course (Primary Key).
  - **Title:** Name of the course (NOT NULL, must be provided).
  - **Credits:** The credit value of the course, restricted by a CHECK constraint to be greater than 0.
  - **DepartmentID:** Identifies the department offering the course, linked via a Foreign Key to the Department table.

SQL Queries: Course Table

- Constraints used:
  - **PRIMARY KEY** – Ensures each course is uniquely identified by CourseID.
  - **NOT NULL** – Ensures Title is always provided.
  - **CHECK (Credits > 0)** – Enforces positive credit values.
  - **FOREIGN KEY** – Establishes a relationship with Department, ensuring that each course belongs to a valid department.

SQL Queries: Course Table

```
-- DDL Commands to CREATE the COURSE Table
CREATE TABLE Course (
    CourseID VARCHAR(10) PRIMARY KEY, Title VARCHAR(100) NOT NULL,
    Credits INT CHECK (Credits>0), DepartmentID VARCHAR(10),
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

```
-- use the DESC (or DESCRIBE) command in
 MySQL to display its structure
DESC Course;
```

Output

Field	Type	Null	Key	Default	Extra
CourseID	varchar(10)	NO	PRI	NULL	
Title	varchar(100)	NO		NULL	
Credits	int	YES		NULL	
DepartmentID	varchar(10)	YES	MUL	NULL	

SQL Queries: Course Table

```
-- DML Commands to INSERT values into the Course Table
INSERT INTO Course VALUES ('CS101', 'Data Structures', 4, 'D01'),('CS102', 'DBMS', 3, 'D01'),
('CS103', 'Operating Systems', 3, 'D01'),
('EC101', 'Digital Circuits', 3, 'D02'),
('EC102', 'Analog Electronics', 3, 'D02');
```

```
-- DQL Command to SELECT or display
all data in the COURSE table
SELECT * FROM Course ;
```

Output

CourseID	Title	Credits	DepartmentID
CS101	Data Structures	4	D01
CS102	DBMS	3	D01
CS103	Operating Systems	3	D01
EC101	Digital Circuits	3	D02
EC102	Analog Electronics	3	D02

SQL Queries: Professor Table

- The **Professor table** stores **details of professors working** in **various departments**.
  - **ProfessorID:** Unique identifier for each professor (Primary Key).
  - **Name:** Full name of the professor (NOT NULL, must be provided).
  - **Email:** Email address of the professor, must be unique across all records.
  - **DepartmentID:** Identifies the department the professor belongs to, linked via a Foreign Key to the Department table.

SQL Queries: Professor Table

- Constraints used:
  - **PRIMARY KEY** – Ensures each professor is uniquely identified by ProfessorID.
  - **NOT NULL** – Ensures Name is always provided.
  - **UNIQUE** – Enforces that each Email is unique to avoid duplicates.
  - **FOREIGN KEY** – Establishes a relationship with Department, ensuring each professor is assigned to a valid department.

SQL Queries: Professor Table

```
-- DDL Commands to CREATE the PROFESSOR Table
CREATE TABLE Professor (
    ProfessorID VARCHAR(10) PRIMARY KEY, Name VARCHAR(50) NOT NULL,
    Email VARCHAR(50) UNIQUE, DepartmentID VARCHAR(10),
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

-- use the DESC (or DESCRIBE) command in MySQL to display its structure

```
DESC Professor;
```

Output

Field	Type	Null	Key	Default	Extra
ProfessorID	varchar(10)	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
Email	varchar(50)	YES	UNI	NULL	
DepartmentID	varchar(10)	YES	MUL	NULL	

SQL Queries: Professor Table

```
-- DML Commands to INSERT values into the PROFESSOR Table
INSERT INTO Professor VALUES ('P01', 'Dr. Kumar', 'kumar@csuniv.edu', 'D01'), ('P02', 'Dr. Meena',
'meena@csuniv.edu', 'D01'), ('P03', 'Dr. Raj', 'raj@eceuniv.edu', 'D02'), ('P04', 'Dr. Leela',
'leela@eceuniv.edu', 'D02'), ('P05', 'Dr. Ravi', 'ravi@csuniv.edu', 'D01');
```

```
-- DQL Command to SELECT or display
all data in the PROFESSOR table
SELECT * FROM Professor ;
```

Output

ProfessorID	Name	Email	DepartmentID
P01	Dr. Kumar	kumar@csuniv.edu	D01
P02	Dr. Meena	meena@csuniv.edu	D01
P03	Dr. Raj	raj@eceuniv.edu	D02
P04	Dr. Leela	leela@eceuniv.edu	D02
P05	Dr. Ravi	ravi@csuniv.edu	D01

SQL Queries: Enrollment Table

- The **Enrollment table** records the **enrollment details of students** in **various courses** for specific semesters.
  - **EnrollmentID:** Auto-incremented unique identifier for each enrollment record (Primary Key).
  - **StudentID:** Identifies the student who is enrolled, linked via a Foreign Key to the Student table.
  - **CourseID:** Identifies the course in which the student is enrolled, linked via a Foreign Key to the Course table.
  - **Semester:** The semester during which the student is enrolled in the course.
  - **Grade:** The grade received by the student for the course (e.g., A+, B, etc.).

SQL Queries: Enrollment Table

- Constraints used:
  - **PRIMARY KEY** – Ensures each enrollment record is uniquely identified by EnrollmentID.
  - **UNIQUE** (StudentID, CourseID, Semester) – Prevents a student from being enrolled more than once in the same course during the same semester.
  - **FOREIGN KEY** – Establishes relationships with Student and Course tables, ensuring that enrollments reference valid students and courses.

SQL Queries: Enrollment Table

```
-- DDL Commands to CREATE the ENROLLMENT Table
CREATE TABLE Enrollment (
EnrollmentID INT AUTO_INCREMENT PRIMARY KEY,
StudentID VARCHAR(10), CourseID VARCHAR(10), Semester VARCHAR(20), Grade
VARCHAR(2),
UNIQUE (StudentID, CourseID, Semester), -- to prevent duplicate enrollments
FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
FOREIGN KEY (CourseID) REFERENCES Course(CourseID));
```

SQL Queries: Enrollment Table

-- use the DESC (or DESCRIBE) command in MySQL to display its structure

```
DESC Enrollment;
```

Output

Field	Type	Null	Key	Default	Extra
EnrollmentID	int	NO	PRI	NULL	auto_increment
StudentID	varchar(10)	YES	MUL	NULL	
CourseID	varchar(10)	YES	MUL	NULL	
Semester	varchar(20)	YES		NULL	
Grade	varchar(2)	YES		NULL	

SQL Queries: Enrollment Table

```
-- DML Commands to INSERT values into the ENROLLMENT Table
INSERT INTO Enrollment  (StudentID, CourseID, Semester, Grade) VALUES
('CS001', 'CS101', 'Odd Sem 2024', 'A'), ('CS002', 'CS101', 'Odd Sem 2024', 'B'), ('CS003', 'CS101',
'Odd Sem 2024', 'B'), ('CS001', 'CS102', 'Even Sem 2024', 'A'),('CS002', 'CS102', 'Even Sem 2024',
'B'),('CS003', 'CS103', 'Even Sem 2024', 'B'),('EC001', 'EC101', 'Odd Sem 2024', 'A'),('EC004',
'EC101', 'Odd Sem 2024', 'C'),('EC001', 'EC102', 'Even Sem 2024', 'B');
```

```
-- DQL Command to SELECT or display
all data in the ENROLLMENT table
SELECT * FROM Enrollment ;
```

Output

EnrollmentID	StudentID	CourseID	Semester	Grade
1	CS001	CS101	Odd Sem 2024	A
2	CS002	CS101	Odd Sem 2024	B
3	CS003	CS101	Odd Sem 2024	B
4	CS001	CS102	Even Sem 2024	A
5	CS002	CS102	Even Sem 2024	B
6	CS003	CS103	Even Sem 2024	B
7	EC001	EC101	Odd Sem 2024	A
8	EC004	EC101	Odd Sem 2024	C
9	EC001	EC102	Even Sem 2024	B

SQL Queries: Prerequisite Table

- The **Prerequisite table** defines the **prerequisite relationships** between **courses, specifying which courses** must be completed before enrolling in another course.
  - **CourseID:** Identifies the course that requires a prerequisite, linked via a Foreign Key to the Course table.
  - **PrerequisiteID:** Identifies the prerequisite course, also linked via a Foreign Key to the Course table.

SQL Queries: Prerequisite Table

- Constraints used:
  - **PRIMARY KEY** (CourseID, PrerequisiteID) – Ensures that each course–prerequisite pair is unique, preventing duplicate entries for the same relationship.
  - **FOREIGN KEY** (CourseID) – Links the course to the Course table, ensuring the main course exists.
  - **FOREIGN KEY** (PrerequisiteID) – Links the prerequisite course to the Course table, ensuring it is a valid course.

SQL Queries: Prerequisite Table

```
-- DDL Commands to CREATE the PREREQUISITE Table
CREATE TABLE Prerequisite (
CourseID VARCHAR(10), PrerequisiteID VARCHAR(10),
PRIMARY KEY (CourseID, PrerequisiteID),
FOREIGN KEY (CourseID) REFERENCES Course(CourseID),
FOREIGN KEY (PrerequisiteID) REFERENCES Course(CourseID));
```

-- use the DESC (or DESCRIBE) command in MySQL to display its structure

```
DESC Prerequisite;
```

Output

Field	Type	Null	Key	Default	Extra
CourseID	varchar(10)	NO	PRI	NULL	
PrerequisiteID	varchar(10)	NO	PRI	NULL	

SQL Queries: Prerequisite Table

```
-- DML Commands to INSERT values into the PREREQUISITE Table
INSERT INTO Prerequisite (CourseID, PrerequisiteID) VALUES
('CS102', 'CS101'),
('CS103', 'CS101'),
('EC102', 'EC101');
```

```
-- DQL Command to SELECT or display
all data in the PREREQUISITE table
SELECT * FROM Prerequisite ;
```

Output

CourseID	PrerequisiteID
CS102	CS101
CS103	CS101
EC102	EC101

SQL Queries: Teaches table

- The **Teaches table** records which **professors are assigned** to **teach specific courses**.
  - **ProfessorID:** Identifies the professor teaching the course, linked via a Foreign Key to the Professor table.
  - **CourseID:** Identifies the course being taught, linked via a Foreign Key to the Course table.
- **Constraints used:**
  - **PRIMARY KEY** (ProfessorID, CourseID) – Ensures each professor–course assignment is unique, preventing duplicate records for the same teaching assignment.
  - **FOREIGN KEY** (ProfessorID) – Ensures that the professor exists in the Professor table.
  - **FOREIGN KEY** (CourseID) – Ensures that the course exists in the Course table.

SQL Queries: Teaches table

```
-- DDL Commands to CREATE the TEACHES Table
CREATE TABLE Teaches (
ProfessorID VARCHAR(10), CourseID VARCHAR(10),
PRIMARY KEY (ProfessorID, CourseID),
FOREIGN KEY (ProfessorID) REFERENCES Professor(ProfessorID),
FOREIGN KEY (CourseID) REFERENCES Course(CourseID));
```

```
-- use the DESC (or DESCRIBE) command in
 MySQL to display its structure
DESC Teaches;
```

Output

Field	Type	Null	Key	Default	Extra
ProfessorID	varchar(10)	NO	PRI	NULL	
CourseID	varchar(10)	NO	PRI	NULL	

SQL Queries: Teaches table

```
-- DML Commands to INSERT values into the Teaches Table
INSERT INTO Teaches (ProfessorID, CourseID) VALUES
('P01', 'CS101'),('P02', 'CS102'),('P05', 'CS103'),
('P03', 'EC101'),('P04', 'EC102');
```

```
-- DQL Command to SELECT or display
all data in the TEACHES table
SELECT * FROM Teaches ;
```

Output

ProfessorID	CourseID
P01	CS101
P02	CS102
P05	CS103
P03	EC101
P04	EC102

Database Querying - MySQL Clauses

Introduction

- In MySQL, **clauses** are **keywords** or instructions that **define conditions, filters, ordering, grouping, or limits in a SQL statement.**
- A clause usually works alongside a **SQL command** like SELECT, INSERT, UPDATE, or DELETE to refine what data is retrieved, inserted, updated, or deleted.

Why Do We Need Clauses?

- Without clauses, **SQL commands** would be **too general** and **often return all data**, which is **inefficient**.
- Clauses help by:
  - **Filtering** → Only return records that meet certain conditions (WHERE, HAVING)
  - **Sorting** → Arrange results in a desired order (ORDER BY)
  - **Grouping** → Aggregate similar data (GROUP BY)
  - **Limiting** → Restrict number of results (LIMIT).
- They make queries **precise, faster, and more meaningful.**

Categories of Clauses

Clause	Purpose	Why We Need It	Example (Using University Database)
WHERE	Filters rows	Avoids unnecessary data retrieval	Get only Computer Science students → WHERE Major = 'Computer Science'
ORDER BY	Sorts results	Makes data easier to read/analyze	Sort students by name → ORDER BY Name ASC
GROUP BY	Groups rows for aggregation	Enables summaries & analytics	Count students per major → GROUP BY Major
HAVING	Filters after grouping	Filters aggregate results	Majors with more than 10 students → HAVING COUNT(*) > 10

Categories of Clauses

Clause	Purpose	Why We Need It	Example (Using University Database)
LIMIT	Restricts output rows	Improves performance & control	Show top 5 students → LIMIT 5
DISTINCT	Removes duplicates	Avoids repeated results	List unique majors → SELECT DISTINCT Major

Categories of Clauses

- Let us understand the Categories of **CLAUSE** with **STUDENT TABLE ,COURSE TABLE and DEPARTMENT TABLE** from **UNIVERSITY DATABASE**

```
-- STUDENT Table
CREATE TABLE Student (
    StudentID VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(50) UNIQUE, DOB DATE,
    Major VARCHAR(15) NOT NULL CHECK (Major IN ('CS', 'IT', 'Mechanical', 'Civil', 'ECE', 'EEE'))
);
```

OUTPUT

StudentID	Name	Email	DOB	Major
CS001	Alice Johnson	alice@example.com	2002-05-10	CS
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS003	Ethan White	ethan@example.com	2000-12-30	CS
EC001	Charlie Brown	charlie@example.com	2003-01-20	ECE
EC004	Diana Green	diana@example.com	2002-11-03	ECE

Categories of Clauses

```
-- DEPARTMENT Table
CREATE TABLE Department (
    DepartmentID VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Building VARCHAR(50) DEFAULT 'MAIN BLOCK'
);
```

Output

DepartmentID	Name	Building
D01	Computer Science	CS BLOCK
D02	Electronics & Communication	ECE BLOCK

Categories of Clauses

```
-- COURSE Table
CREATE TABLE Course (
    CourseID VARCHAR(10) PRIMARY KEY, Title VARCHAR(100) NOT NULL,
    Credits INT CHECK (Credits>0), DepartmentID VARCHAR(10),
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

Output

CourseID	Title	Credits	DepartmentID
CS101	Data Structures	4	D01
CS102	DBMS	3	D01
CS103	Operating Systems	3	D01
EC101	Digital Circuits	3	D02
EC102	Analog Electronics	3	D02

WHERE CLAUSE

- The **WHERE CLAUSE** filters **certain records** that **meet the conditions** mentioned in the query.
- It is evaluated second after the FROM clause.
- We can use the **following operators** with the WHERE clause.

Operator	Description
=	Equal
>=	Greater than equal to
<=	less than equal to
>	Greater than
<	Less than
<>	Not equal
BETWEEN	Between a range
Like	Search for a pattern

WHERE CLAUSE : Example

- **WHERE clause** is **not a mandatory clause** of SQL DML statements, but if you want to **limit the number of rows** to be affected by your DML query or the **number of rows to return** from your select statement, then you need to use the Where Clause in MySQL.

Example #1: Retrieve the student's details belong to major ‘CS’

```
SELECT * from student WHERE Major='cs';
```

OUTPUT

StudentID	Name	Email	DOB	Major
CS001	Alice Johnson	alice@example.com	2002-05-10	CS
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS003	Ethan White	ethan@example.com	2000-12-30	CS

WHERE CLAUSE : Example

Example #2: Filters only rows where the Email starts with "a"

SELECT * from student WHERE Email LIKE ('a%')

OUTPUT

StudentID	Name	Email	DOB	Major
CS001	Alice Johnson	alice@example.com	2002-05-10	CS

- **WHERE clause** can combine multiple conditions using **AND, OR, NOT**

Example #3: filters students whose Email starts with 'a' (Email LIKE 'a%') OR their Major is 'ECE'

OUTPUT

StudentID	Name	Email	DOB	Major
CS001	Alice Johnson	alice@example.com	2002-05-10	CS
EC001	Charlie Brown	charlie@example.com	2003-01-20	ECE
EC004	Diana Green	diana@example.com	2002-11-03	ECE

SELECT * from student WHERE Email LIKE ('a%') OR Major='ECE';

ORDER BY CLAUSE

- The **Order By Clause** is used for **sorting the data** either in **ascending or descending** order of a query based on a specified column or list of columns.
- By **default**, the Order By Clause in MySQL will sort the data in **ascending** order.
- If you want to arrange the data in descending order, then you must have to use the **DESC** keyword.
- The Order By Clause can be **applied to any data type** column in the table.
- This clause will **arrange the data temporarily** but not in the permanent store.
- The Order By Clause can only be used in Select Statements.

Syntax:

```
SELECT expressions FROM tables [WHERE conditions] ORDER BY expression [ASC | DESC];
```

ORDER BY CLAUSE

Parameters used in the **above syntax** are as follows:

1. **Expression**: The columns or calculations that we want to retrieve.
2. **Tables**: The tables from which we want to retrieve the records. There should be at least one table specified in the FROM clause.
3. **WHERE Conditions:** It is optional. The conditions must be met for the records to be selected by the query.
4. **ASC**: It is optional. If you want to sort the result set in ascending order of the expression, then you need to use ASC.
5. **DESC**: It is optional. If you want to sort the result set in descending order by expression, then you need to the DESC keyword.

ORDER BY CLAUSE : Example

Example #1: Retrieves all columns and all rows from the Student table and Sorts the results by the Major column in descending order (Z to A).

SELECT * from student ORDER BY Major DESC;

OUTPUT

StudentID	Name	Email	DOB	Major
EC001	Charlie Brown	charlie@example.com	2003-01-20	ECE
EC004	Diana Green	diana@example.com	2002-11-03	ECE
CS001	Alice Johnson	alice@example.com	2002-05-10	CS
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS003	Ethan White	ethan@example.com	2000-12-30	CS

- If we are using **where clause** and **order by clause** in a **single query**, then **first where clause gets executed and then order by clause gets executed**

Example #2: Filters the Student table to include only students whose Major is 'cs' and Sorts the filtered results by Name in descending order (Z to A).

SELECT * from student where Major= 'cs' ORDER BY Name DESC;

OUTPUT

StudentID	Name	Email	DOB	Major
CS003	Ethan White	ethan@example.com	2000-12-30	CS
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS001	Alice Johnson	alice@example.com	2002-05-10	CS

ORDER BY CLAUSE : Example

Example #1: Retrieves all columns and all rows from the Student table and Sorts the results by the Major column in descending order (Z to A).

SELECT * from student ORDER BY Major DESC;

OUTPUT

StudentID	Name	Email	DOB	Major
EC001	Charlie Brown	charlie@example.com	2003-01-20	ECE
EC004	Diana Green	diana@example.com	2002-11-03	ECE
CS001	Alice Johnson	alice@example.com	2002-05-10	CS
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS003	Ethan White	ethan@example.com	2000-12-30	CS

- If we are using **where clause** and **order by clause** in a **single query**, then **first where clause gets executed and then order by clause gets executed**

Example #2: Filters the Student table to include only students whose Major is 'cs' and Sorts the filtered results by Name in descending order (Z to A).

SELECT * from student where Major= 'cs' ORDER BY Name DESC;

OUTPUT

StudentID	Name	Email	DOB	Major
CS003	Ethan White	ethan@example.com	2000-12-30	CS
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS001	Alice Johnson	alice@example.com	2002-05-10	CS

ORDER BY CLAUSE : Example

Example #3: Retrieves the columns StudentID, Name, and Major from the Student table and Sorts the results by the second column in the SELECT list, which is Name, in descending order (Z to A).

SELECT StudentID, Name, Major from student ORDER BY 2 DESC;

OUTPUT

StudentID	Name	Major
CS003	Ethan White	CS
EC004	Diana Green	ECE
EC001	Charlie Brown	ECE
CS002	Bob Smith	CS
CS001	Alice Johnson	CS

Explanation of ORDER BY 2:

- The **number 2** in **ORDER BY 2** means "**order by the second selected column.**"
- Here, the columns selected are: StudentID, Name, Major. So, **ORDER BY 2** is equivalent to **ORDER BY Name.**

ORDER BY CLAUSE : Example

- When we have **multiple columns in order by clause**, the **data first gets arranged based on the first column** and if any **duplicate values** are there in the **first column** then it will **take the support of the second column** for the **arrangement** or **else the second column will not be used**.

Example #4: ORDER BY Clause with both ASC and DESC

```
SELECT * from student where year(DOB)>2000 ORDER
BY major ASC, name DESC;
```

OUTPUT

StudentID	Name	Email	DOB	Major
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS001	Alice Johnson	alice@example.com	2002-05-10	CS
EC004	Diana Green	diana@example.com	2002-11-03	ECE
EC001	Charlie Brown	charlie@example.com	2003-01-20	ECE

LIMIT CLAUSE

- The **LIMIT** clause is used to **restrict the number of rows** returned **by a query**.
- This clause is often combined with **ORDER BY** to **fetch a specific subset of rows**.

```
-- Syntax
SELECT column1, column2, ... FROM table_name
[WHERE condition]
[ORDER BY column_name ASC|DESC]
LIMIT [offset,] row_count;
```

- **Parameters** used in the **above syntax** are as **f**ollows:
  - **row_count**: The number of rows to return.
  - **offset** *(optional)*: The starting position to retrieve rows (0-based index). If omitted, it defaults to 0.

LIMIT CLAUSE: Example

Example #1: Fetch the first 3 students

```
SELECT * FROM Student LIMIT 3;
```

OUTPUT

StudentID	Name	Email	DOB	Major
CS001	Alice Johnson	alice@example.com	2002-05-10	CS
CS002	Bob Smith	bob@example.com	2001-08-15	CS
CS003	Ethan White	ethan@example.com	2000-12-30	CS

Example #2: Fetch students starting from 3rd row (Pagination: skip 2 rows, fetch next 3):

```
SELECT * FROM Student LIMIT 2, 3;
```

OUTPUT

StudentID	Name	Email	DOB	Major
CS003	Ethan White	ethan@example.com	2000-12-30	CS
EC001	Charlie Brown	charlie@example.com	2003-01-20	ECE
EC004	Diana Green	diana@example.com	2002-11-03	ECE

GROUP BY CLAUSE

- The **GROUP BY CLAUSE** is used to **divide similar types** of **records** or data as a **group** and then return.
- If we use group by clause in the query then we **should use** groupings/**aggregate functions** such as **count(), sum(), max(), min(), and avg() functions**.
- That means first **Group By clause** is used to **divide similar types of data** as a **group** and then an **aggregate function is applied** to **each group** to get the required results.
- When we implement group by clause **first the data of the table** will be **divided into the separate group** as per the column and later **aggregate function** will **execute** on **each group's data to get the result.**

GROUP BY CLAUSE

Syntax:

```
SELECT expression1, expression2, expression_n, aggregate_function (expression)
FROM tables [WHERE conditions] GROUP BY expression1, expression2, expression_n;
```

In above syntax, the parameter used in GROUP BY Clause in MySQL:

1. **expression1, expression2, expression_n:** The expressions that are not encapsulated within an aggregate function must be included in the GROUP BY clause.
2. **aggregate_function:** The aggregate function is nothing but such as SUM, COUNT, MIN, MAX, or AVG functions that we should use while we are using the Group by Clause in MySQL.
3. **Tables:** Tables are name of the table or tables from which we want to retrieve the data.
4. **WHERE conditions (optional):** Use if you want to retrieve the data based on some conditions

GROUP BY CLAUSE : Example

Example #1: Count the number of courses in each department

```
SELECT DepartmentID, Count(*) as Total_Courses
FROM Course GROUP BY DepartmentID;
```

OUTPUT

DepartmentID	Total_Courses
D01	3
D02	2
D03	2

Example #2: Find the maximum credit in each department

```
SELECT DepartmentID, Min(Credits) as
Minimum_Credit FROM Course
GROUP BY DepartmentID;
```

OUTPUT

DepartmentID	Minimum_Credit
D01	3
D02	3
D03	2

GROUP BY CLAUSE : Example

- When we use **multiple columns** in a **group by clause first data** in the table is divided based on the **first column** of the group by clause and then each group is subdivided based on the **second column** of the group by clause and then the group function is applied on each inner group to get the result.
- When the aggregate function is applied to a group it returns only a single value but **each group can return a value.**

Example #3: Count the number of courses in each credit per department

```
SELECT DepartmentID, Credits, COUNT(*) as CreditCount
FROM Course GROUP BY DepartmentID, Credits;
```

OUTPUT

DepartmentID	Credits	CreditCount
D01	4	1
D01	3	2
D02	3	2
D03	5	1
D03	2	1

HAVING CLAUSE

- The **HAVING CLAUSE** is **used for restricting** or you can say **filtering the data** just like the where clause in MySQL.
- So, the Having Clause in MySQL is an **additional filter** that is **applied to the result set**.
- Logically, the having clause **filters the rows** from the **intermediate result** set that is built by using the **FROM, WHERE, or GROUP BY clauses in the SELECT statement.**
- The Having clause is typically **used** with a **GROUP BY clause**. That means it is **used in combination with a GROUP BY clause** to **restrict the number of groups** to be returned by satisfying the condition which is specified using the having clause.

HAVING CLAUSE

Syntax:

-- The Having Clause Condition is used to add a further condition that can be applied only to the aggregated results to restrict the number of groups to be returned.

```
SELECT expression1, expression2, expression_n,
       aggregate_function (expression)
FROM tables
[WHERE conditions]
GROUP BY expression1, expression2, expression_n
HAVING having_condition;
```

HAVING CLAUSE: Example

- The **Where clause** in MySQL is used to **filter the rows before aggregation**, whereas the **Having clause** in MySQL is used to **filter the groups after aggregations.**

Example #1 : Filter rows using WHERE

```
SELECT DepartmentID, SUM(Credits) as TotalCredit FROM Course WHERE DepartmentID = 'D01'
GROUP BY DepartmentID;
```

Example #2 : Filter rows using HAVING Clause

```
SELECT DepartmentID, SUM(Credits) as TotalCredit FROM Course GROUP BY DepartmentID
HAVING DepartmentID = 'D01' ;
```

HAVING CLAUSE: Example

Example #3: Groups the courses by their DepartmentID, Calculates the total credits (SUM(Credits)) for each department and Returns only those departments where the total credits are greater than 6.

```
SELECT DepartmentID, SUM(Credits) as TotalCredit FROM
Course GROUP BY DepartmentID HAVING SUM(Credits)>6;
```

OUTPUT

DepartmentID	TotalCredit
D01	10
D03	7

HAVING CLAUSE: Example

Example #4: The query groups courses by DepartmentID, calculates the maximum and minimum credits per department, and returns only those departments where the minimum course credit exceeds 2.

```
SELECT DepartmentID, Max(Credits) as Max_Credit, Min(Credits) as
Min_Credit FROM Course GROUP BY DepartmentID HAVING
Min(Credits)>2;
```

OUTPUT

DepartmentID	Max_Credit	Min_Credit
D01	4	3
D02	3	3

DISTINCT CLAUSE

- The **DISTINCT clause** is used in a **SELECT statement** to **remove duplicate rows** from the **result set**.
- It ensures that **each row in the output is unique** based on the columns you select.

DISTINCT CLAUSE : Example

- Suppose you want to **list all unique building names** where **departments are located**.
- **Sample Data (imagine):**

DepartmentID	Name	Building
D01	Computer Sci	MAIN BLOCK
D02	Mechanical	BLOCK B
D03	Electrical	MAIN BLOCK
D04	Civil	BLOCK A

```
-- Example : Query without DISTINCT
SELECT Building FROM Department;
-- Result: Retrieves all building names with the duplicates
```

DISTINCT CLAUSE : Example

```
-- Example : Query with DISTINCT
SELECT DISTINCT Building FROM Department;
-- Result: Retrieves all building names without the duplicates
```

Quiz

1) Which SQL clause is used to filter rows based on a specified condition?

a) ORDER BY

b) GROUP BY

c) WHERE

d) HAVING

Answer : Option c)

Quiz

2) Which clause is used to sort the result set in ascending or descending order?

a) ORDER BY

b) GROUP BY

c) DISTINCT

d) HAVING

Answer : Option a)

Quiz

3) In which clause do you specify the condition for filtering aggregated data?

a) WHERE

b) LIMIT

c) DISTINCT

d) HAVING

Answer : Option d)

Quiz

4) What does the DISTINCT clause do in a SELECT statement?

a) Sorts the result set

b) Limits the number of rows returned

c) Removes duplicate rows from the result set

d) Filters rows based on a condition

Answer : Option c)

Quiz

5) Which SQL function is used to find the total number of rows in a table?

a) SUM()

b) COUNT()

c) AVG()

d) MAX()

Answer : Option b)


THANK YOU
SmartCliff
Career Mobility Solutions